

1. Transport Layer Core Role & Addressing

L4 Protocol Engine

- Process-to-Process Delivery:** Network layer delivers host-to-host (IP). Transport layer delivers process-to-process using **Port Addresses**.
- Socket Address:** Socket = IP Address + Port Number (e.g., 192.168.1.10:80). Uniquely identifies an active socket endpoint globally.
- Port Range (16-bit: 0 to 65,535):**
 - Well-Known (0–1023):* Assigned by IANA (HTTP:80, HTTPS:443, DNS:53, FTP:20/21, SSH:22, SMTP:25).
 - Registered (1024–49151):* Registered for proprietary apps (MySQL:3306, RDP:3389).
 - Dynamic / Ephemeral (49152–65535):* Assigned temporarily by OS to client processes.
- Multiplexing & Demultiplexing:**
 - Multiplexing (Sender):* Gathers data from multiple sockets, adds L4 headers, and passes down to IP layer.
 - Demultiplexing (Receiver):* Reads Dst Port, routes segment to matching socket buffer.

2. TCP Foundations & Byte-Stream Model

RFC 793

- Core Attributes:** Connection-Oriented, Point-to-Point (unicast only), Reliable (ACK + Retransmit), Full-Duplex (simultaneous 2-way), Stream-Oriented.
- Byte-Stream Service:** TCP treats data as a continuous sequence of individual bytes, NOT discrete packets. Transferred in chunks called **Segments**.
- Circular Buffers:** Send buffer holds bytes waiting for transmission / ACK. Receive buffer reorders out-of-order bytes and absorbs bursts.
- Sequence & ACK Numbers:**
 - Sequence Number (32-bit):* Identifies byte number of 1st data byte in segment: $Seq = ISN + 1st_byte_offset$.
 - ACK Number (32-bit):* Cumulative acknowledgment: tells sender "I have safely received all bytes up to N-1; I expect byte N next".

3. TCP Segment Header Architecture

20 to 60 Bytes

Field Name	Width	Function & Rules
Source / Dest Port	16 b each	Calling & receiving process port numbers.
Sequence Number	32 bits	First data byte's byte-stream number.
Acknowledgment No	32 bits	Next expected byte number (valid if ACK=1).
Header Length (HLEN)	4 bits	No. of 4-byte words. Formula: Bytes = HLEN × 4. Minimum: 5 (20B), Max: 15 (60B).
Reserved	6 bits	Must be 0 (used for NS, CWR, ECE flags).
Control Flags	6 bits	URG, ACK, PSH, RST, SYN, FIN (1 bit each).
Window Size (rwnd)	16 bits	Receiver buffer credit in bytes (Flow control).
Checksum	16 bits	Mandatory! Covers Header + Data + Pseudo-Header.
Urgent Pointer	16 bits	Offset from Seq No to last urgent byte (if URG=1).
Options + Padding	0–40 B	MSS (Max Segment Size), Window Scale, SACK, Timestamps. Padded to 32-bit boundary.

4. TCP 6 Control Flags Summary

1-Bit Triggers

- URG:** Urgent pointer field is valid; segment contains high-priority out-of-band data.
- ACK:** Acknowledgment field is valid (set in all segments post initial SYN).
- PSH:** Receiver must push data immediately to application without buffering.
- RST:** Resets connection abruptly due to error, port mismatch, or host crash.
- SYN:** Synchronizes sequence numbers during connection establishment. Consumes 1 sequence number.
- FIN:** Sender has finished sending data; initiates teardown. Consumes 1 sequence number.

5. Connection Establishment (3-Way Handshake)

SYN-ACK

- Step 1 (Client → Server):** SYN=1, ACK=0, Seq=ISN_c. Client enters SYN-SENT. Consumes 1 Seq No (no payload).
- Step 2 (Server → Client):** SYN=1, ACK=1, Seq=ISN_s, Ack=ISN_c+1. Server allocates buffers/TCB and enters SYN-RCVD. Consumes 1 Seq No.
- Step 3 (Client → Server):** SYN=0, ACK=1, Seq=ISN_c+1, Ack=ISN_s+1. Both enter ESTABLISHED. Can carry data payload.
- SYN Flooding Attack (DoS):** Attacker spams spoofed SYN packets; server creates half-open connections in backlog queue until memory runs out.
- Defense — SYN Cookies:** Server encodes state into $ISN_s = Hash(src, dst, secret)$ without allocating RAM until valid 3rd ACK arrives.

6. Connection Termination & Half-Close

FIN Handshake

- Normal 4-Way Handshake:**
 - Client sends FIN=1, Seq=u → Enters FIN-WAIT-1.
 - Server sends ACK=1, Ack=u+1 → Enters CLOSE-WAIT; Client enters FIN-WAIT-2.
 - Server finishes residual data, sends FIN=1, Seq=v, Ack=u+1 → Enters LAST-ACK.
 - Client sends ACK=1, Ack=v+1 → Enters TIME-WAIT for **2MSL**; Server enters CLOSED.
- Piggybacked 3-Way Teardown:** If Server has no pending data, it combines Step 2 & 3: sends FIN+ACK directly.
- Half-Close:** Client stops sending data but can still receive data indefinitely until server finishes its transmission.
- TIME-WAIT State (2MSL):** Client waits $2 \times \text{text{MSL}}$ (~120s) to:
 - Ensure final ACK reaches server (resend if lost).
 - Allow duplicate / lingering segments to expire in internet routers before reuse.

7. TCP Data Transfer, PUSH & URGENT Mechanics

Stream Control

- **Normal Data Stream:** Segment carries data bytes. Sequence number is the index of first byte. Acknowledgment number confirms receipt of previous bytes.
- **Pushing Data (PSH Flag):**
 - Normal behavior: TCP sender/receiver buffers data for transmission efficiency.
 - With `PSH=1`: Sender flushes output buffer instantly; Receiver flushes input buffer directly to app layer without waiting for a full MSS block.
- **Urgent Data (URG Flag & Pointer):**
 - Used for out-of-band signals (e.g., Telnet/SSH Ctrl+C, abort commands).
 - Urgent Pointer = 16-bit offset added to Sequence Number pointing to byte immediately after last urgent byte.

8. Flow Control & Credit Sliding Window

Buffer Matching

- **Core Objective:** Prevent fast sender from overflowing slow receiver's buffer.
- **Effective Window Formula:**

$$\text{Effective Window} = \min(\text{rwnd}, \text{cwnd})$$

`rwnd` = Receiver Advertised Window; `cwnd` = Congestion Window.
- **Window States (Sender Perspective):**
 - *Opening:* Right wall expands rightward (receiver advertises larger buffer).
 - *Closing:* Left wall slides rightward (ACK acknowledges sent bytes).
 - *Shrinking:* Right wall moves leftward (strongly discouraged by RFC 793).
- **Zero-Window Deadlock:** Receiver advertises `rwnd=0`. Later, buffer frees up and receiver sends `rwnd=2000` update, which gets LOST in transit! Both sides wait indefinitely!
- **Deadlock Solution (Persistence Timer):** When `rwnd=0`, sender starts Persistence Timer. On expiry, sends a 1-byte **Window Probe Segment**. Receiver replies with current `rwnd`.
- **Silly Window Syndrome (SWS):**
 - *Receiver-driven:* Receiver advertises tiny 1-byte window. Fixed by **Clark's Solution**: Do not advertise window until $\text{rwnd} \geq \min(\text{MSS}, \text{Buffer}/2)$.
 - *Sender-driven:* App writes 1 byte at a time. Fixed by **Nagle's Algorithm**: Send 1st byte immediately; buffer subsequent tiny bytes until full MSS or previous ACK arrives.

9. TCP Error Control & Retransmission Engine

Reliability

- **Three Error Detection Mechanisms:**
 1. *Checksum:* 16-bit 1s complement sum over header, data, pseudo-header. If corrupt, dropped silently.
 2. *Acknowledgments:* Cumulative ACKs. Confirms all contiguous bytes up to ACK-1.
 3. *Retransmission Timer (RTO):* Resends segment if no ACK received before RTO expires.
- **Retransmission Triggers:**
 - *RTO Timeout:* Heavy loss/congestion indicator $\implies$ drops `cwnd = 1 MSS`.
 - *3 Duplicate ACKs (Fast Retransmit):* Segment lost but network still delivering packets $\implies$ resends missing segment without waiting for RTO timer.
- **Out-of-Order Segments:** TCP does not drop out-of-order bytes; it buffers them and sends duplicate ACK of last contiguous in-order byte.

10. Dynamic RTO Formulas (Jacobson & Karn)

Mathematical Core

- **Sample RTT:** Time measured between sending segment and receiving ACK.
- **Smoothed RTT (SRTT):** Weighted moving average:

$$\text{SRTT}_{\text{new}} = (1 - \alpha) \cdot \text{SRTT}_{\text{old}} + \alpha \cdot \text{SampleRTT}$$

$\alpha = 0.125$
- **RTT Variation (RTTVAR):** Jitter/variance metric:

$$\text{RTTVAR}_{\text{new}} = (1 - \beta) \cdot \text{RTTVAR}_{\text{old}} + \beta \cdot |\text{SampleRTT} - \text{SRTT}|$$

$\beta = 0.25$
- **Retransmission Timeout (RTO):**

$$\text{RTO} = \text{SRTT} + 4 \cdot \text{RTTVAR}$$
- **Karn's Algorithm & Exponential Backoff:**
 - *Rule 1:* NEVER update SampleRTT for retransmitted segments (ACK ambiguity).
 - *Rule 2 (Timer Backoff):* On retransmission, double RTO: $\text{RTO}_{\text{new}} = 2 \times \text{RTO}_{\text{old}}$ (exponential backoff up to 64s). Reset to normal formula only after a fresh, non-retransmitted ACK.

11. Flow Control vs Congestion Control Distinction

Key Difference

Feature	Flow Control	Congestion Control
Target Entity	Protects Receiver Buffer	Protects Global Network Routers
Governed By	Advertised Window (<code>rwnd</code>)	Congestion Window (<code>cwnd</code>)
Metric Origin	Receiver feedback (ACK field)	Packet loss & delay inferred by sender
Equation	$\text{Actual Transmission Window} = \min(\text{rwnd}, \text{cwnd})$	

12. TCP Congestion Control Algorithms

AIMD Architecture

- Additive Increase Multiplicative Decrease (AIMD):** Increases bandwidth gently; cuts drastically on overload to maintain network stability.
- Phase 1: Slow Start (\$cwnd < ssthresh\$):**
 - Initial $cwnd = 1 \text{ MSS}$.
 - For every ACK received: $cwnd = cwnd + 1 \text{ MSS}$ $\implies$ Doubles every RTT (2^n exponential growth).
 - Transitions to Congestion Avoidance when $cwnd \geq ssthresh$.
- Phase 2: Congestion Avoidance (\$cwnd \geq ssthresh\$):**
 - Increases by 1 MSS per RTT: $cwnd = cwnd + (1 / cwnd)$ per ACK $\implies$ Linear growth.
- Loss Detection & Recovery:**
 - Case A: RTO Timeout (Severe Congestion):**
 $ssthresh = cwnd / 2$, $cwnd = 1 \text{ MSS}$ $\implies$ Restart Slow Start.
 - Case B: 3 Duplicate ACKs (Mild Congestion / Isolated Drop):**
TCP Tahoe: $ssthresh = cwnd / 2$, $cwnd = 1 \text{ MSS}$ (Restart Slow Start).
TCP Reno: $ssthresh = cwnd / 2$, $cwnd = ssthresh + 3 \text{ MSS}$ (Fast Recovery $\implies$ skips Slow Start).

13. TCP Timers Architecture & 11-State FSM

RFC 793

- Four Operational Timers:**
 - RTO Timer:** Dynamic retransmission timer per unacknowledged segment.
 - Persistence Timer:** Prevents zero-window deadlock; sends periodic probes.
 - Keepalive Timer:** 2 hours idle detection. Sends 10 probes @ 75s intervals before resetting.
 - TIME-WAIT Timer:** Runs for 2MSL (~120s) before client socket is recycled.
- TCP 11 States:** CLOSED, LISTEN, SYN-SENT, SYN-RCVD, ESTABLISHED, FIN-WAIT-1, FIN-WAIT-2, CLOSE-WAIT, CLOSING, LAST-ACK, TIME-WAIT.

14. User Datagram Protocol (UDP RFC 768)

Lightweight

- Properties:** Connectionless, Unreliable, Message-Oriented (preserves application message boundaries), No Congestion Control, Minimal Overhead.
- 8-Byte Header:** Src Port (16b) | Dst Port (16b) | Length (16b) | Checksum (16b).
- Length Field:** Header (8B) + Data bytes. Min value = 8.
- Pseudo-Header (12 Bytes):** IP Source (4B), IP Dest (4B), Zero (1B), Protocol (1B = 17), UDP Length (2B). Used exclusively during 1s-complement checksum computation to detect cross-host delivery errors. Checksum is optional in IPv4 (0 if disabled), mandatory in IPv6.

15. Master Comparison: TCP vs UDP

Exam Favorite Table

Parameter	TCP (RFC 793)	UDP (RFC 768)
Connection	Connection-Oriented (3-way)	Connectionless
Reliability	Reliable (ACK + Retransmit)	Unreliable (Best Effort)
Header Size	20–60 Bytes (Heavy)	8 Bytes fixed (Lightweight)
Data Delivery	Byte Stream (No boundaries)	Message / Datagram-oriented
Flow / Congestion	Yes (Sliding window + AIMD)	None (Blasts at app rate)
Speed / Latency	Slower (Handshake overhead)	Ultra-fast, lowest latency
Communication	Unicast point-to-point only	Unicast, Multicast & Broadcast
Key Applications	HTTP/S, FTP, SMTP, SSH	DNS, DHCP, VoIP, Gaming, RTP

16. Quality of Service (QoS) & Traffic Shaping

L4 Optimization

- 4 QoS Metrics:** Delay/Latency (propagation+queuing), Jitter (delay variance), Bandwidth/Throughput, Packet Loss rate.
 - Packet Scheduling Disciplines:**
 - FIFO:** First-in first-out; zero fairness, prone to bufferbloat.
 - Priority Queuing:** High-priority traffic served first; can starve low-priority.
 - Weighted Fair Queuing (WFQ):** Allocates guaranteed bandwidth ratio per queue.
 - Leaky Bucket vs Token Bucket Traffic Shapers:**
 - Leaky Bucket:** Eliminates bursts; outputs at a rigid constant rate. Overflows drop packets.
 - Token Bucket:** Generates tokens at rate \$R\$. Allows bursts up to capacity \$C\$ at maximum rate \$M\$.
- Max Burst Duration:** $T = C / (M - R)$
 C = Token Capacity, M = Max Burst Transmission Rate, R = Token Arrival Rate.

17. AKTU Numerical & Formula Cheat Bank

High-Yield Formulas

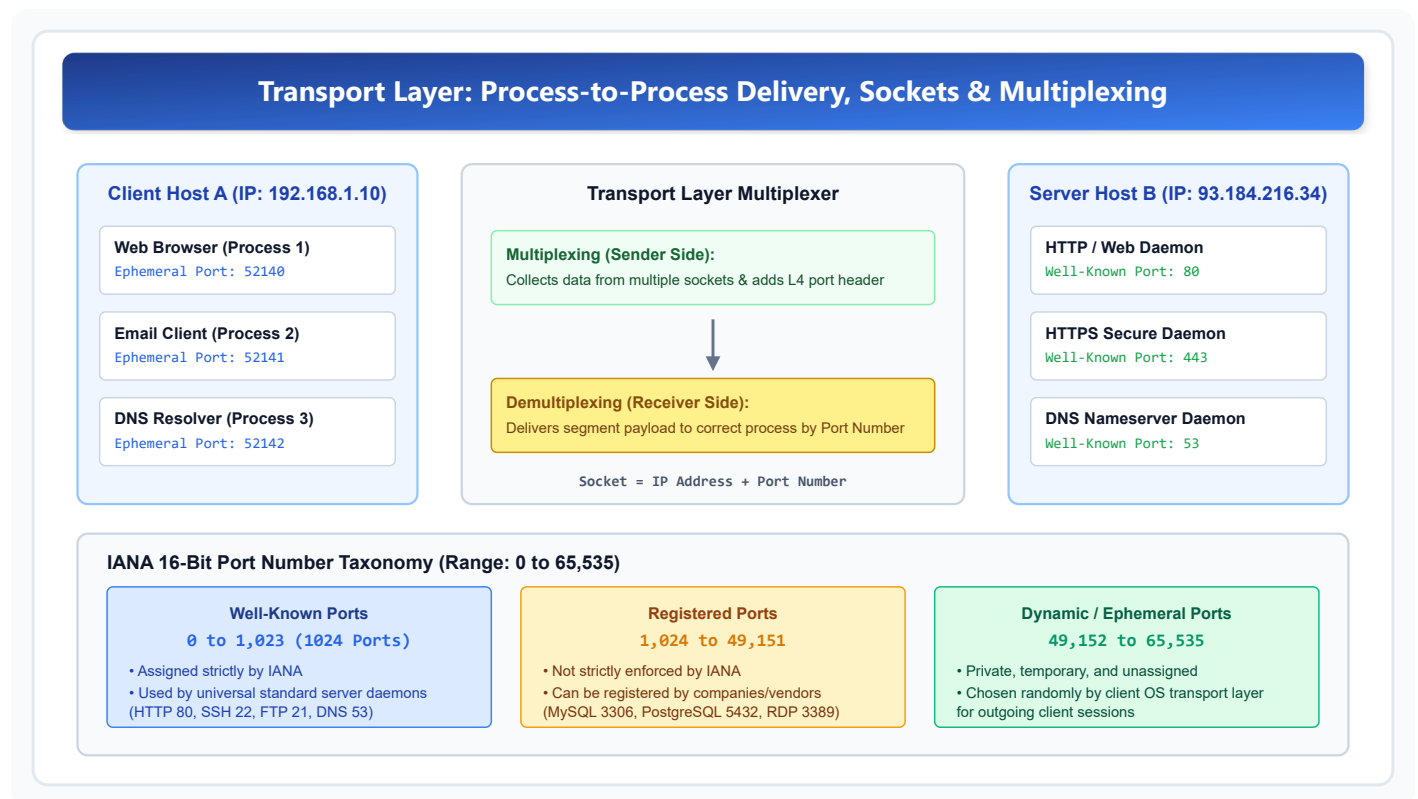
- TCP Header Length (Bytes) = $HLEN \times 4$. If $HLEN = 0111_2$ (7) $\implies 7 \times 4 = 28$ Bytes (20B base + 8B options).
- $MSS = MTU - (IP_Header + TCP_Header) = 1500 - 40 = 1460$ Bytes.
- Slow Start Rounds to reach Threshold: $n = \log_2(ssthresh / cwnd_0)$.
- Window Utilization / Efficiency: $\eta = \frac{W}{cwnd} \cdot \text{MSS}$ $\implies \frac{\text{Bandwidth} \times \text{RTT}}$.
- Token Bucket Burst: $\text{Max Volume Transmitted} = C + R \cdot T = M \cdot T$.

Module 01: Transport Layer Services and Port Addressing

Unit 4: Transport Layer

Process-to-Process Delivery

End-to-End Communication Mechanics, 16-Bit Port Hierarchy, Socket Addresses, Multiplexing vs Demultiplexing, and Connection Paradigms.



1. Port Number Taxonomy (IANA Standard)

Well-Known Ports: 0 to 1,023 (Universal servers: HTTP 80, DNS 53, SSH 22)

Registered Ports: 1,024 to 49,151 (Vendor daemons: MySQL 3306, RDP 3389)

Dynamic / Ephemeral Ports: 49,152 to 65,535 (Temporary client ports chosen by OS)

Socket Address = IP Address : Port Number (e.g., 192.168.1.10:52140)

2. 5-Tuple Connection Uniqueness

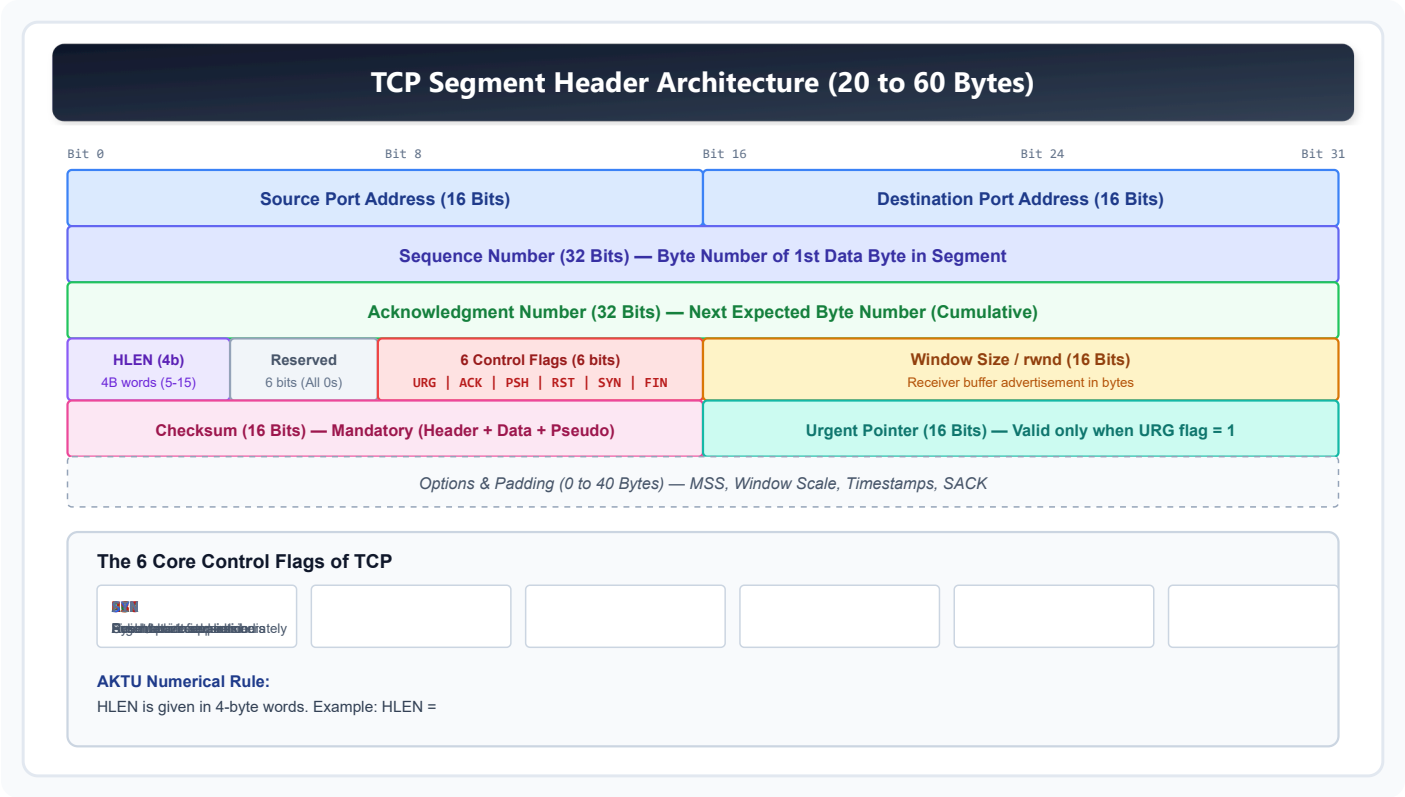
Every active connection across the Internet is uniquely differentiated by the 5-Tuple: **{Source IP, Source Port, Dest IP, Dest Port, Protocol}**.

Module 02: TCP Foundations and Segment Header Format

Unit 4: Transport Layer

TCP Header Architecture

Byte-Stream Service, 20-60 Byte Header Field Specifications, 6 Control Flags, Window Advertisement, and Solved AKTU Hex Dump Numericals.



5 $\implies 5 \times 4 = 20$ ext{ Bytes}\$. Maximum HLEN = 15 $\implies 15 \times 4 = 60$ ext{ Bytes}\$. Sequence numbers consume 1 sequence count per SYN and FIN flag even if carrying 0 data bytes!

1. 14 Header Fields Summary

HLEN Equation: Header Length in Bytes = HLEN (4 bits) * 4 (Min = 20B, Max = 60B)

Control Flags (6 bits): URG | ACK | PSH | RST | SYN | FIN

Window Size (16 bits): rwnd buffer advertisement (Max standard = 65,535 Bytes)

2. Solved AKTU Hex Dump Key Findings

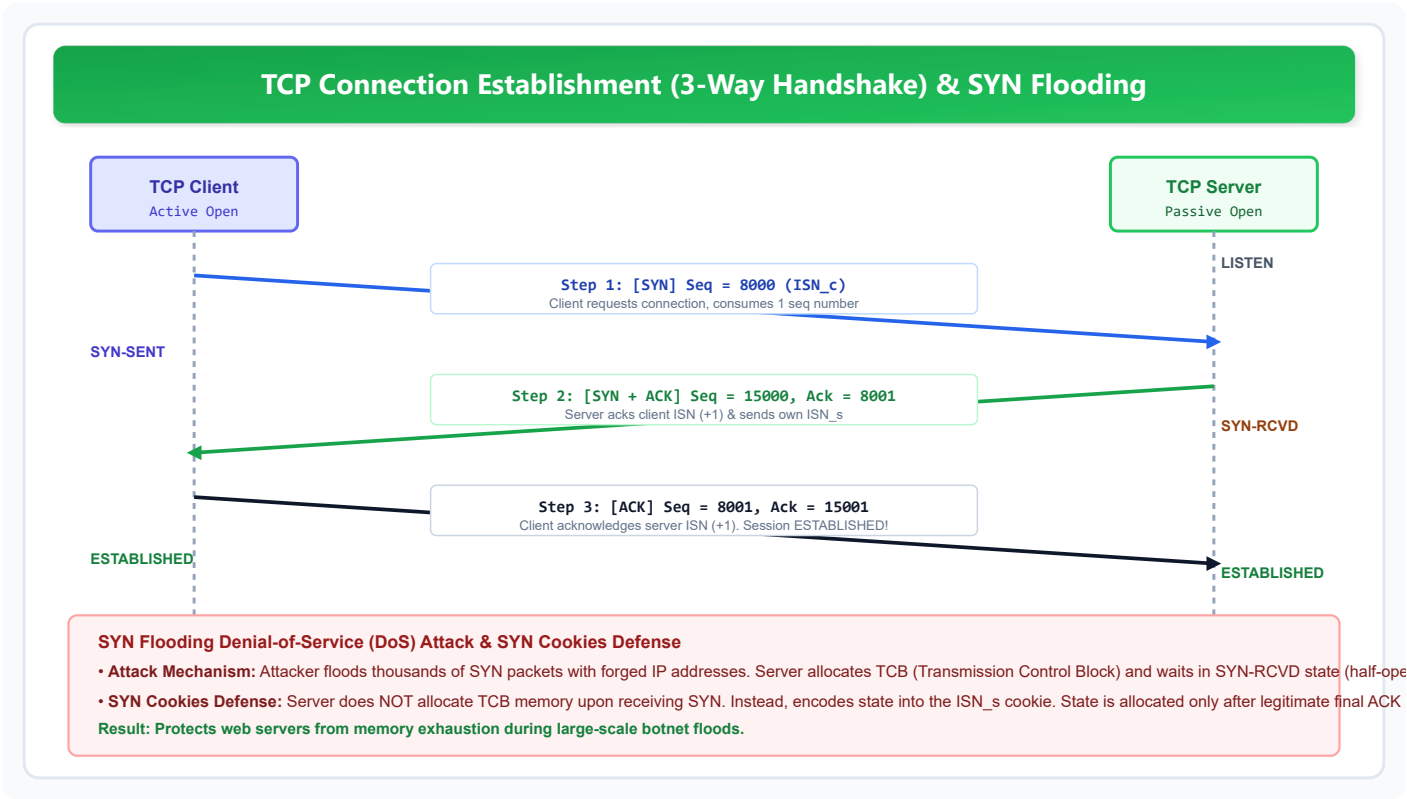
Field	Hex Chunk	Decimal / Meaning
Source Port	05 32	1330 (Ephemeral Client Port)
Destination Port	00 17	23 (Telnet Server)
Sequence Number	00 00 00 01	1 (Initial Sequence Number)
Header Length	5 (High nibble of byte 12)	5 * 4 = 20 Bytes (No options)
Active Flags	02 (Byte 13)	SYN = 1 (Connection Request)
Window Size	07 FF	2047 Bytes advertised

Module 03: TCP Connection Establishment and SYN Flooding

Unit 4: Transport Layer

3-Way Handshake & Security

TCP Three-Way Handshake Sequence, State Transitions, Sequence Consumption Rules, SYN Flooding DoS Attacks, and Cryptographic SYN Cookies Defense.



1. 3-Way Handshake Sequence Breakdown

Step	Sender -> Receiver	Active Flags	Seq / Ack Equations	State Transition
1	Client -> Server	SYN = 1	Seq = ISN_c (consumes 1 seq)	Client: CLOSED -> SYN-SENT

2	Server -> Client	SYN = 1, ACK = 1	Seq = ISN _s , Ack = ISN _c + 1	Server: LISTEN -> SYN-RCVD
3	Client -> Server	ACK = 1	Seq = ISN _c + 1, Ack = ISN _s + 1	Both enter ESTABLISHED state

2. SYN Cookies Defense Principle

Zero State Allocation: Server does not allocate memory on SYN arrival.

$ISN_s = \text{HMAC}(\text{Client_IP}, \text{Client_Port}, \text{Server_IP}, \text{Server_Port}, \text{Counter}, \text{Key})$

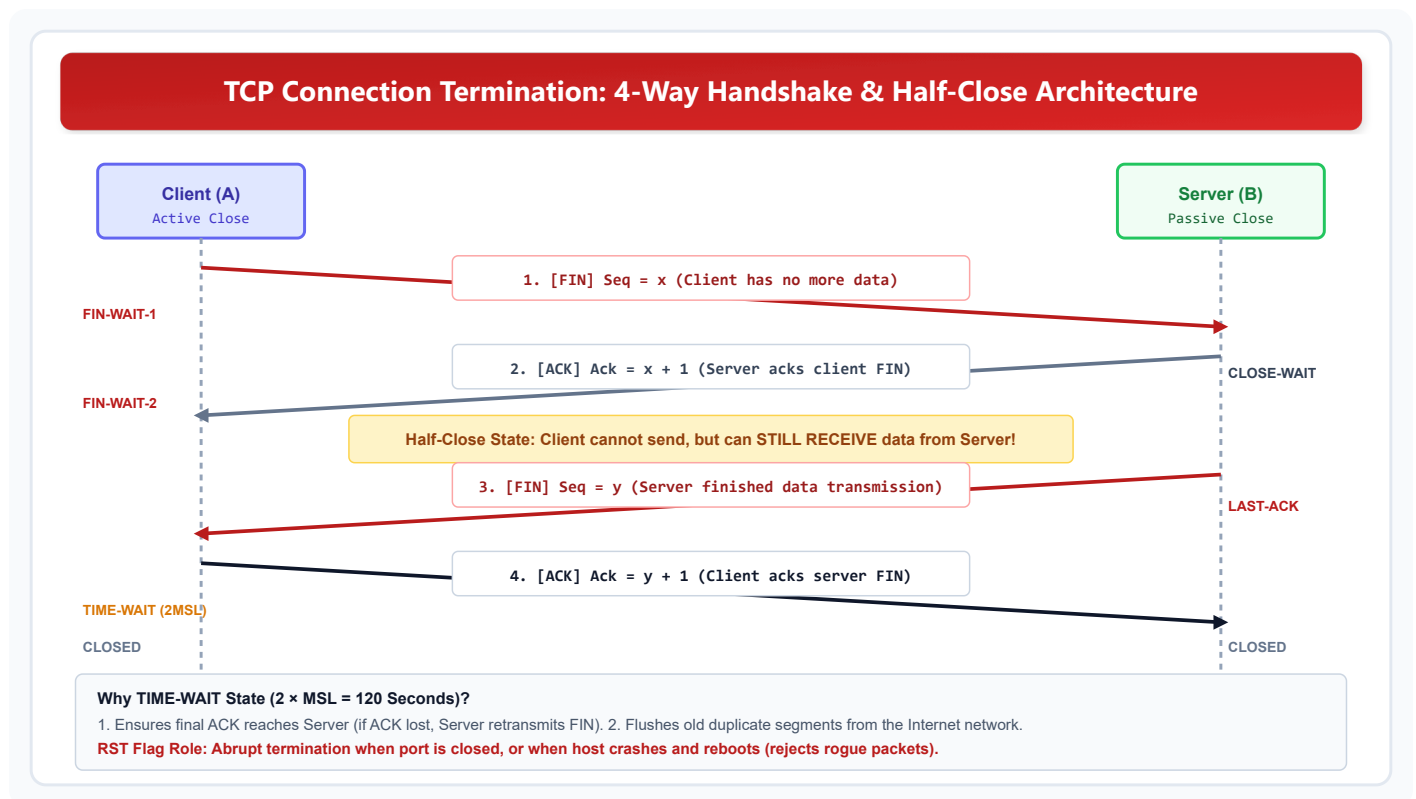
State is instantiated only upon reception of verified final ACK!

Module 04: TCP Connection Termination and Half-Close

Unit 4: Transport Layer

Termination & State Dynamics

Four-Way Handshake with FIN & ACK, Half-Close Architecture, TIME-WAIT (2MSL) State Engineering, and Reset (RST) Flag Usage.



1. 4-Way Handshake Steps

Step	Direction	Flag	Equations	States
1	Client -> Server	FIN = 1	Seq = x	Client: FIN-WAIT-1
2	Server -> Client	ACK = 1	Ack = x + 1	Client: FIN-WAIT-2, Server: CLOSE-WAIT

3	Server -> Client	FIN = 1	Seq = y	Server: LAST-ACK
4	Client -> Server	ACK = 1	Ack = y + 1	Server: CLOSED, Client: TIME-WAIT (2MSL)

2. TIME-WAIT (2MSL) Rationale

Reason 1: Allows retransmitting final ACK if Server's FIN times out.

Reason 2: Prevents old duplicate segments from corrupting future connections.

RST Flag: Abrupt teardown when connecting to closed port or during crash recovery.

Module 05: TCP Data Transfer, Pushing, and Urgent Data

Unit 4: Transport Layer

Interactive & Urgent Streams

TCP Normal Buffering Stream Model, PUSH (PSH) Immediate Transmission, URG Flag & 16-Bit Urgent Pointer Offset Math, and Out-of-Band Control Signals.

TCP Pushing Data (PSH Flag) & Urgent Data (URG Flag / Pointer)

1. Pushing Data Mechanics (PSH Flag = 1)

Default TCP Buffering Behavior:

- Sending TCP waits until buffer accumulates MSS bytes.
- Receiving TCP buffers segments until application requests read.

High delay — Unacceptable for interactive shells (Telnet / SSH)!

With PSH Flag SET (PSH = 1):

- Sending TCP flushes send buffer immediately without waiting.
- Segment transmitted across network instantly.
- Receiving TCP delivers data directly to application layer, completely bypassing receive buffer delay!

Typical Interactive Use Case:

User types a single character in SSH / Telnet session.
PSH flag forces instant echo back to terminal screen!

2. Urgent Data & Urgent Pointer (URG Flag = 1)

TCP Data Stream Buffer Layout:

Normal Data

URGENT DATA

Ctrl+C / Abort Signal

Normal Data

↑
Urgent Pointer

Urgent Pointer Calculation Formula:

End of Urgent Byte = Seq Number + Urgent Pointer - 1

- URG flag = 1 indicates 16-bit Urgent Pointer field is valid.
- Urgent data is extracted & processed out-of-order ahead of queue!

Real-World Out-of-Band Abort Case:

While downloading a massive 10 GB file via FTP, user presses
Ctrl + C (Abort command).
Without URG, abort command waits behind thousands of queued bytes!
With URG, server stops transmission instantly!

1. PSH vs URG Core Comparison

Feature	Pushing Data (PSH = 1)	Urgent Data (URG = 1)
Primary Goal	Eliminates buffer waiting latency	Sends out-of-band interrupt / control signals
Queue Order	Processed in-order (FIFO)	Bypasses normal queue (High-priority interrupt)

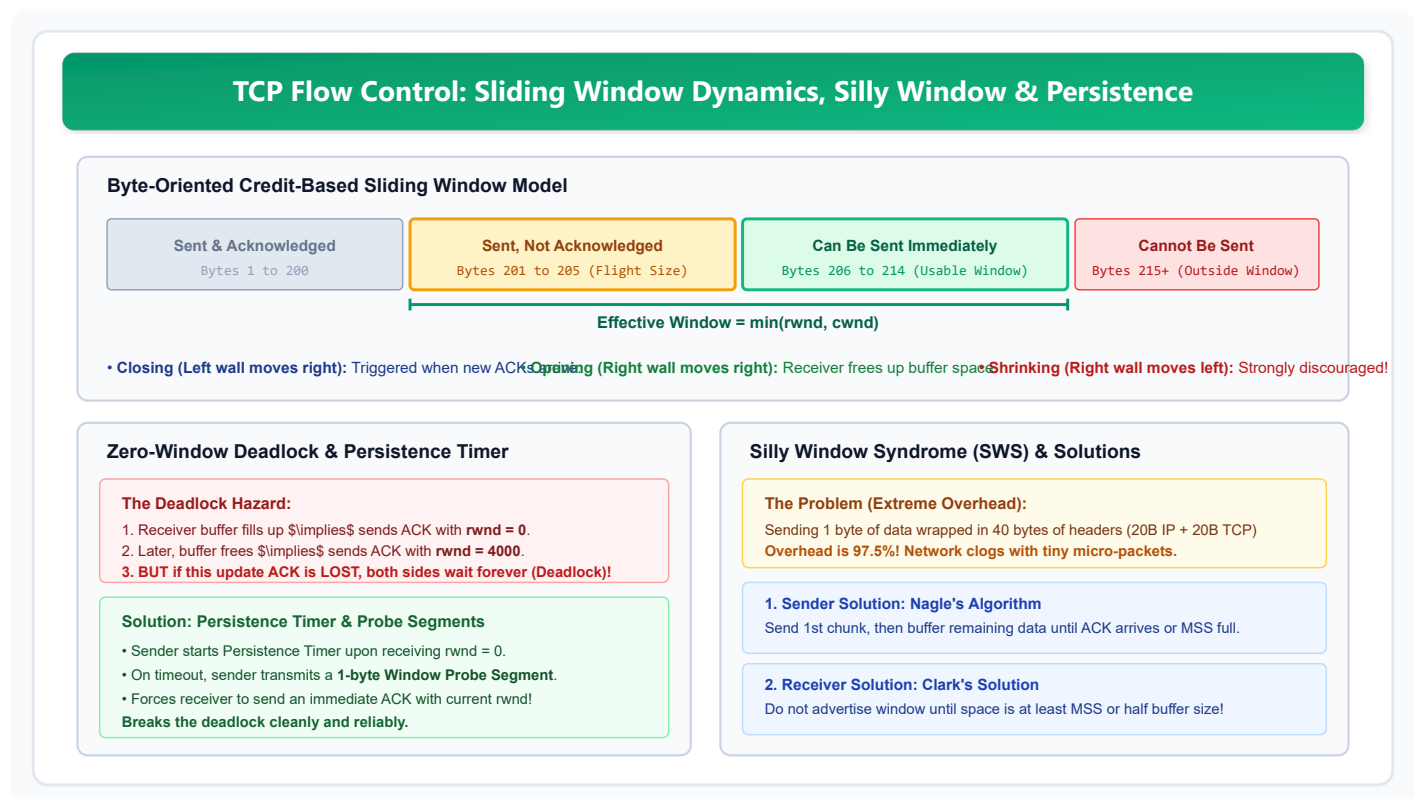
Header Field	PSH bit only	URG bit + 16-bit Urgent Pointer field
Classic Use Case	Interactive Telnet / SSH keystroke echo	User pressing Ctrl+C or FTP abort command

Module 06: TCP Flow Control and Sliding Window Mechanics

Unit 4: Transport Layer

Flow Control & Buffer Management

Credit-Based Byte-Oriented Sliding Window, Window Opening/Closing/Shrinking Dynamics, Zero-Window Deadlock Prevention via Persistence Timer, and Silly Window Syndrome.



1. Sliding Window Wall Dynamics

Window Size Equation: $\text{Effective Window} = \min(\text{rwnd}, \text{cwnd})$

Closing: Left wall moves right upon receiving ACKs for sent bytes.

Opening: Right wall moves right as receiver application drains buffer.

Shrinking: Right wall moves left (Forbidden / strongly discouraged by RFCs).

2. Silly Window Syndrome Mitigations

Technique	Side Applied	Rule Enforced
Nagle's Algorithm	Sender Side	Buffer small chunks until previous ACK returns or MSS accumulated
Clark's Solution	Receiver Side	Do not advertise window until space $\geq$ MSS or $\geq$ half buffer space

Module 07: TCP Error Control and Retransmission Mechanics

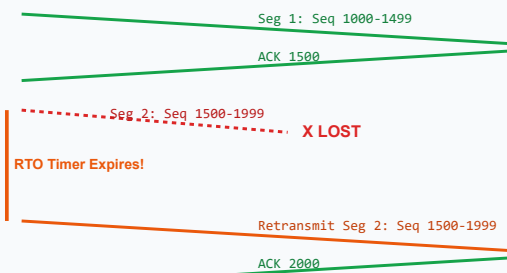
Unit 4: Transport Layer

Reliability & Retransmissions

Checksum Integrity, Cumulative ACKs, Jacobson's Dynamic RTO Mathematical Formula, Karn's Ambiguity Rule, and Three Duplicate ACKs Fast Retransmit.

TCP Error Control: RTO Calculation, Karn's Rule & Fast Retransmit

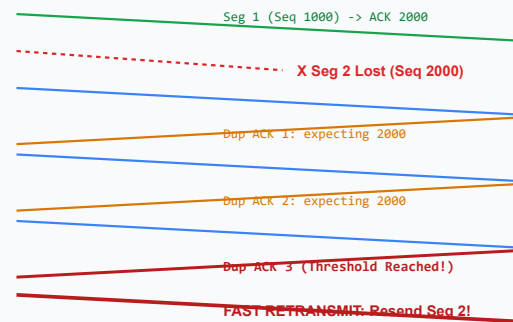
1. Lost Segment & RTO Timer Expiration



Jacobson's RTO Mathematical Formulas:

$\text{EstimatedRTT} = (1 - a) * \text{EstimatedRTT} + a * \text{SampleRTT} \quad (a = 1/8)$
 $\text{DevRTT} = (1 - b) * \text{DevRTT} + b * |\text{SampleRTT} - \text{EstimatedRTT}| \quad (b = 1/4)$
 $\text{RTO} = \text{EstimatedRTT} + 4 * \text{DevRTT}$
Karn's Rule: Do NOT measure SampleRTT for retransmitted segments!

2. Three Duplicate ACKs & Fast Retransmit



Why 3 Duplicate ACKs (Fast Retransmit)?

- Out-of-order delivery can generate 1 or 2 duplicate ACKs.
 - Receiving **3 Duplicate ACKs** guarantees segment loss!
 - Sender resends missing packet **before RTO timer expires**.
- Saves entire seconds of idle waiting time!**

1. Jacobson's Dynamic RTO Equations

$\text{EstimatedRTT} = 0.875 * \text{EstimatedRTT} + 0.125 * \text{SampleRTT}$

$\text{DevRTT} = 0.75 * \text{DevRTT} + 0.25 * |\text{SampleRTT} - \text{EstimatedRTT}|$

$\text{RTO} = \text{EstimatedRTT} + 4 * \text{DevRTT}$

Karn's Rule: Do NOT update RTT estimates for retransmitted segments!

2. Retransmission Triggers

Trigger Type	Detection Condition	Action Taken
RTO Expiration	No ACK arrives before RTO countdown hits 0	Slow retransmission; drops cwnd to 1 MSS
Fast Retransmit	Reception of 3 Duplicate ACKs	Instant retransmission without waiting for RTO!

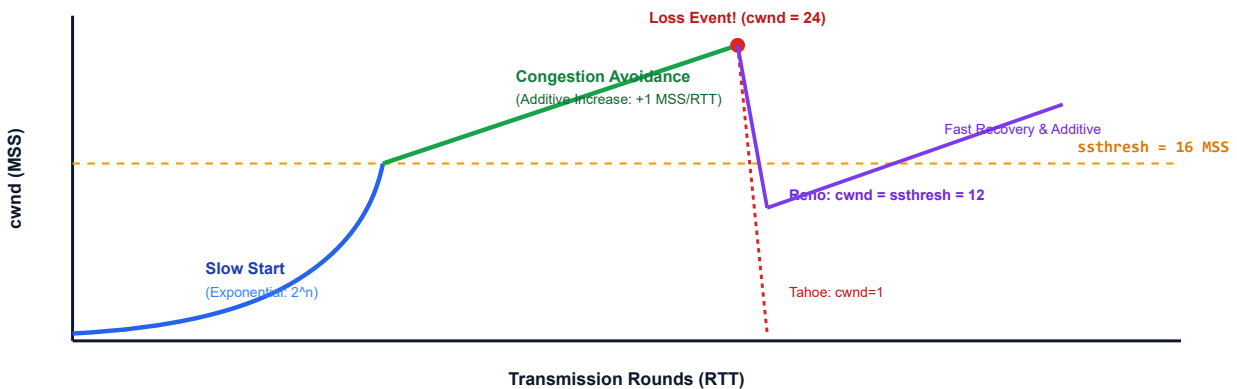
Module 08: TCP Congestion Control: Slow Start and Congestion Avoidance

Unit 4: Transport Layer

Congestion Dynamics & AIMD

Congestion Window (cwnd) vs Receiver Window (rwnd), Slow Start Exponential Ramp-Up, Congestion Avoidance Linear Growth, and AIMD Stability.

TCP Congestion Control: Slow Start, Congestion Avoidance & AIMD Dynamics



Mathematical Formulation of TCP Congestion Window (cwnd)

1. Slow Start & Additive Increase:

$$cwnd = cwnd + 1 \text{ MSS per ACK}$$
$$cwnd = cwnd + 1/cwnd \text{ per ACK}$$

1. Congestion Window Growth Rules

Slow Start: $cwnd = cwnd + 1 \text{ MSS per ACK}$ (Doubles every RTT) until $cwnd \geq ssthresh$.

Congestion Avoidance: $cwnd = cwnd + 1/cwnd \text{ per ACK}$ (Increases linearly by +1 MSS per RTT).

Multiplicative Decrease: Upon congestion, set $ssthresh = cwnd / 2$.

2. AIMD Sawtooth Dynamics

Additive Increase Multiplicative Decrease ensures stable network equilibrium by linearly probing available bandwidth and halving upon packet loss.

Module 09: TCP Congestion Detection: Fast Retransmit and Fast Recovery

Unit 4: Transport Layer

Loss Recovery & Reno Dynamics

Congestion Signals Taxonomy, Severe RTO Timeout vs Mild 3 Duplicate ACKs, TCP Tahoe vs Reno Architectural Comparison, and Fast Recovery State Machine.

TCP Congestion Detection: Timeout vs Fast Retransmit & Fast Recovery

1. Severe Congestion: RTO Timer Expiration

Network Diagnosis: Severe Bottleneck!

- Multiple packets dropped; intermediate router queues fully choked.
- No duplicate ACKs can return to the sender.

Action: Multiplicative decrease to rock bottom!

Mandatory Reaction Steps:

$ssthresh = cwnd / 2$

$cwnd = 1 \text{ MSS}$ (Reset to ground zero)

Resets to Slow Start mode (Exponential increase from 1 MSS).

High throughput penalty — Takes multiple RTTs to recover speed!

TCP Tahoe Philosophy (RFC 1122):

Treats ALL packet losses identically (whether Timeout or Dup ACKs).

Always cuts $ssthresh = cwnd / 2$, drops $cwnd = 1 \text{ MSS}$, and forces Slow Start! Highly conservative and causes throughput collapse.

2. Mild Congestion: 3 Duplicate ACKs (TCP Reno)

Network Diagnosis: Isolated Packet Loss!

- Subsequent segments ARE reaching receiver (triggering Dup ACKs).
- Network pipeline is still flowing, not completely jammed.

Action: Fast Retransmit + Fast Recovery!

TCP Reno Reaction Steps (RFC 2581):

$ssthresh = cwnd / 2$

$cwnd = ssthresh + 3 \text{ MSS}$ (Fast Recovery)

Resends missing segment immediately (Fast Retransmit).

Skips Slow Start completely! Resumes Congestion Avoidance directly.

Modern TCP Variants (NewReno & SACK):

- **TCP NewReno (RFC 6582):** Handles multiple packet drops in 1 window.
- **TCP SACK (RFC 2018):** Selective Acknowledgment allows receiver to report non-contiguous blocks, retransmitting only lost chunks!

1. Severe vs Mild Congestion Response

RTO Timeout: $ssthresh = cwnd / 2$, $cwnd = 1 \text{ MSS}$ (Full reset to Slow Start)

3 Duplicate ACKs: Fast Retransmit missing segment immediately without waiting for RTO.

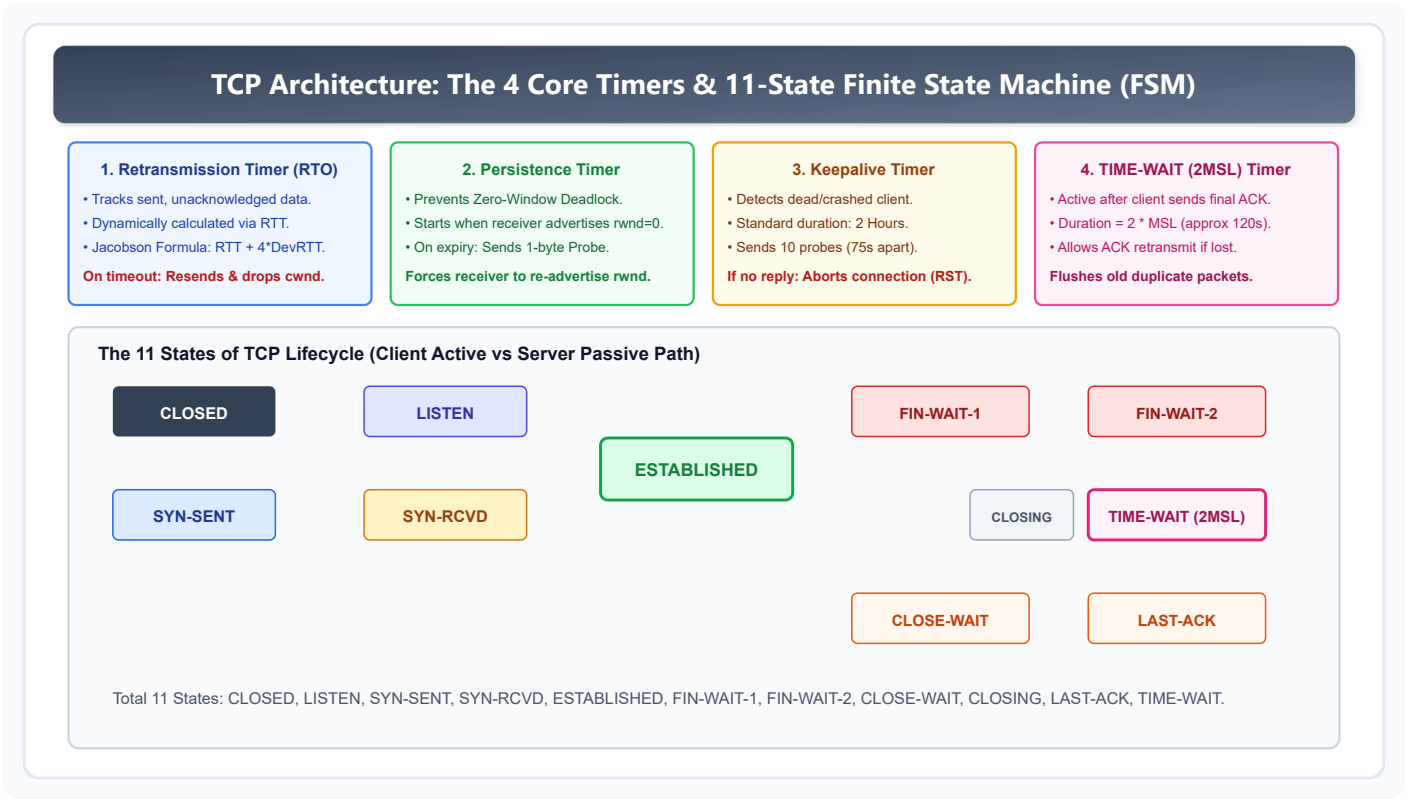
TCP Reno Fast Recovery: $ssthresh = cwnd / 2$, $cwnd = ssthresh$ (Resumes Congestion Avoidance directly).

2. Evolution of TCP Loss Recovery

TCP Version	Loss Detection	Recovery Behavior
Tahoe	Timeout or 3 Dup ACKs	Always cuts cwnd to 1 MSS; enters Slow Start
Reno	3 Duplicate ACKs	Fast Retransmit + Fast Recovery; skips Slow Start
NewReno	Partial ACKs	Recovers multiple packet losses per window without RTO
SACK	Selective ACK blocks	Retransmits exclusively missing packet holes

Module 10: TCP Timers Architecture and State Transition Diagram

The 4 Architectural TCP Timers (RTO, Persistence, Keepalive, TIME-WAIT) and the Complete 11-State Lifecycle Transition State Machine.



Keepalive Timer	2 hours connection idle	Sends 10 probes; aborts dead connection via RST
TIME-WAIT (2MSL)	Final ACK transmitted	Holds port for 120s; flushes stray duplicate packets

2. 11 Lifecycle States Summary

Connection Setup: CLOSED -> (LISTEN) -> SYN-SENT / SYN-RCVD -> ESTABLISHED

Connection Teardown: ESTABLISHED -> FIN-WAIT-1 -> FIN-WAIT-2 -> TIME-WAIT -> CLOSED

Passive Teardown: ESTABLISHED -> CLOSE-WAIT -> LAST-ACK -> CLOSED

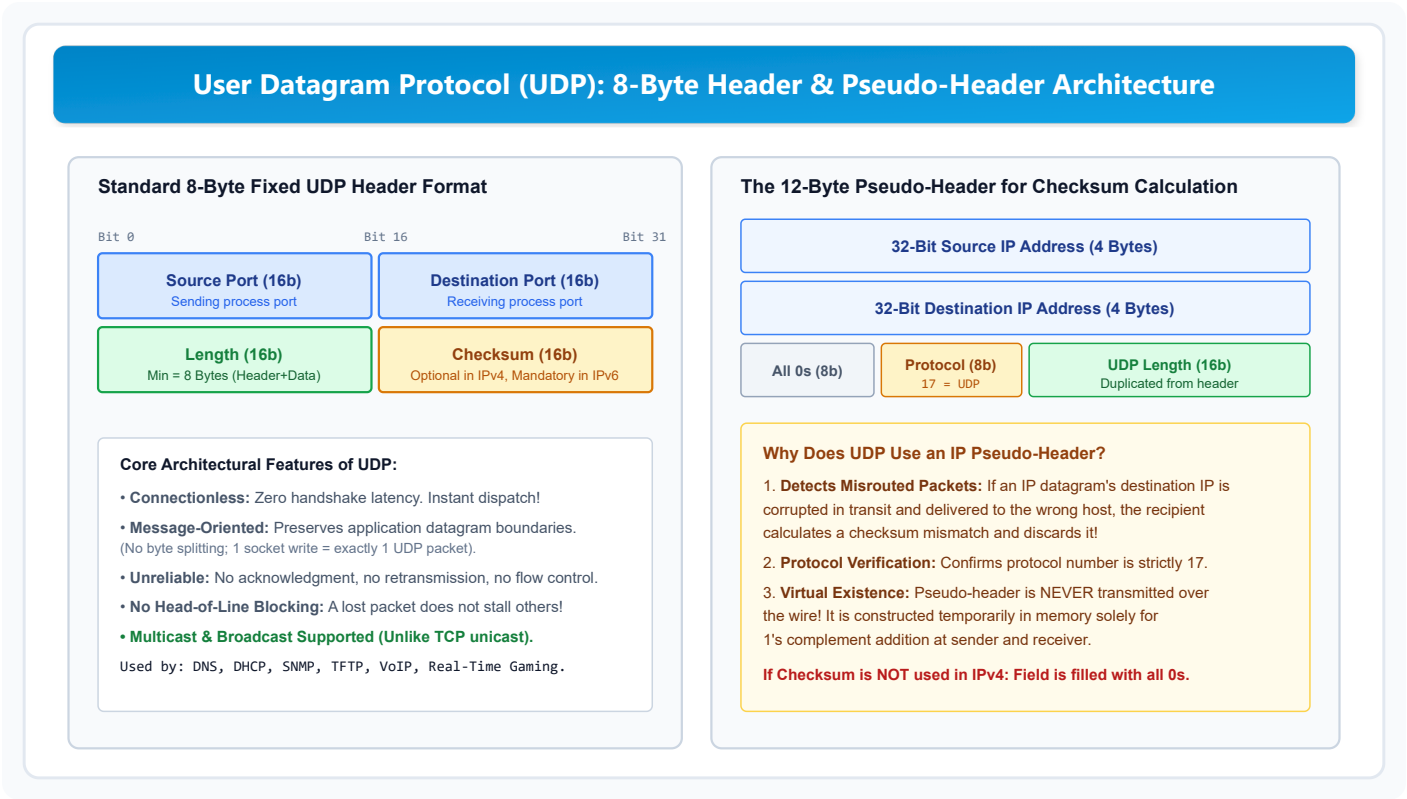
Module 11: User Datagram Protocol (UDP)

Architecture

Unit 4: Transport Layer

Connectionless Datagrams

Minimalist Transport Architecture, 8-Byte Fixed Header Layout, 12-Byte Pseudo-Header Integrity Checksum, Queuing Dynamics, and Applications.



Length	16 bits	Total datagram length in bytes (Min = 8 Bytes)
Checksum	16 bits	Error detection with 12-byte pseudo-header

2. Pseudo-Header Integrity Verification

Pseudo-Header = Source IP (4B) + Dest IP (4B) + 0x00 (1B) + Protocol 17 (1B) + UDP Length (2B)

Virtual Integrity: Detects misrouted IP datagrams delivered to wrong destination hosts!

Module 12: Master Comparison: TCP vs UDP

Exhaustive 12-Point Comparative Matrix, Header Structural Divergence, Performance Metrics, and Practical Protocol Selection Guidelines.

Master Comparison: Transmission Control Protocol (TCP) vs User Datagram Protocol (UDP)

Comparison Parameter	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Connection Paradigm	Connection-Oriented (3-Way Handshake Setup)	Connectionless (Zero handshake setup)
Reliability	100% Reliable (Positive ACK, RTO Retransmissions)	Unreliable (Best-effort delivery; no recovery)
Data Framing Type	Byte-Stream (Continuous bytes, circular buffers)	Message-Oriented (Preserves packet boundaries)
Header Overhead	20 to 60 Bytes (Heavy overhead)	Strictly 8 Bytes Fixed (Extremely lightweight)
Packet Ordering	Guaranteed in-order arrival (Reordered via seq nos)	No ordering guarantee (Arrives out-of-order)
Flow & Congestion Control	Full Flow Control (rwnd) & Congestion (cwnd AIMD)	None (Transmits at application speed)
Transmission Paradigm	Strictly Unicast (One-to-One connection)	Unicast, Multicast & Broadcast Supported!
Speed & Latency	Slower due to handshakes, acks & window limits	Blazing fast; minimum possible latency
Real-World Applications	HTTP/HTTPS (80/443), FTP (21), SSH (22), SMTP	DNS (53), DHCP (67/68), VoIP, TFTP, Gaming

Architectural Decision Rule for Engineering Systems:

- Choose TCP when: Data accuracy and completeness are non-negotiable (Financial transactions, file download, web pages). Losing even 1 byte corrupts the file.
- Choose UDP when: Real-time timeliness matters more than 100% precision (Live video stream, voice call, FPS gaming). Retransmitting a dropped voice frame 500ms late is complete failure.
- Modern Hybrid (QUIC / HTTP/3): Runs on top of UDP to achieve zero handshake RTT, while implementing user-space reliability and stream multiplexing without Head-of-Line block.

1. 12-Point Architectural Divergence

Feature	TCP	UDP
Connection Setup	3-Way Handshake (1.5 RTT setup)	Connectionless (0 RTT instant dispatch)
Reliability	Guaranteed delivery with positive ACKs	Best-effort delivery without recovery
Framing	Continuous byte stream abstraction	Discrete datagram message boundaries
Header Overhead	20 to 60 Bytes	Fixed 8 Bytes

Transmission Scope	Strictly Point-to-Point Unicast	Unicast, Multicast, and Broadcast
---------------------------	---------------------------------	-----------------------------------

Module 13: Quality of Service (QoS) and Traffic Shaping

Unit 4: Transport Layer

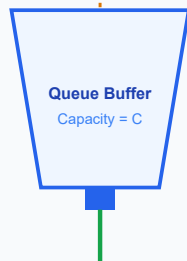
QoS & Traffic Regulation

QoS Performance Parameters, Queue Scheduling Disciplines (FIFO, Priority, WFQ), Leaky Bucket vs Token Bucket Traffic Shaping, and Burst Duration Math.

Quality of Service (QoS): Traffic Shaping & Bucket Algorithms

1. Leaky Bucket Algorithm (Traffic Smoothing)

Bursty Unpredictable Input



Constant Uniform Output Rate (r)

Core Working Principles:

- Inflow can be highly variable and bursty.
- If buffer fills completely, arriving packets are **DISCARDED**.
- Outflow rate is strictly constant (Cannot accommodate bursts!).

2. Token Bucket Algorithm (Allows Controlled Bursts)

- Tokens generated at constant rate R tokens/sec.
- Bucket accumulates up to capacity C tokens.
- Transmitting K bytes requires consuming K tokens.

Maximum Burst Duration Formula (AKTU 10-Marks):

$$\text{Burst Time: } T = C / (M - R)$$

(M = Peak Transmission Rate, R = Token Rate, C = Capacity)

4 Key QoS Performance Metrics:

- **Latency (Delay):** Time for a packet to travel source to destination.
- **Jitter:** Variation in packet arrival delay (crucial for smooth voice/video).
- **Bandwidth / Throughput:** Bits transmitted per unit time.
- **Packet Loss:** Percentage of dropped packets under buffer overflow.

Scheduling Techniques: FIFO | Priority Queuing | WFQ

DiffServ (Differentiated Services): Uses IP ToS byte for per-hop QoS.

1. Token Bucket Burst Mathematics

$$\text{Burst Duration: } T = C / (M - R)$$

$$\text{Total Transmitted Data: } D = M * T = C * M / (M - R)$$

Example: $C = 1\text{MB}$, $M = 50\text{MB/s}$, $R = 10\text{MB/s}$ -> $T = 1 / 40 = 25$ milliseconds (1.25 MB total data)

2. Leaky Bucket vs Token Bucket Comparison

Parameter	Leaky Bucket	Token Bucket
Output Nature	Rigidly constant uniform rate	Allows controlled bursts up to capacity C
Overflow Action	Discards arriving packets	Discards surplus tokens (Zero packet loss)
Suitability	Constant bit-rate streams (Audio/VoIP)	Bursty applications (HTTP web browsing, FTP)

Module 14: Unit 4 AKTU Semester Exam PYQs and Numerical Bank

Unit 4: Transport Layer

Exam Blueprint & Model Solutions

Comprehensive Bank of AKTU Previous Year Questions, 10-Marks Derivations, Hexadecimal Header Dumps, and Mathematical Numerical Solutions.

Unit 4 Transport Layer: AKTU Semester Exam PYQ Blueprint & Formula Bank

High-Frequency 10-Marks Derivations

8. **3-Way Handshake & SYN Flooding** (10 Marks)
Derive the formula for SYN Flood attack based on the given information and QoS.
Example: If the attacker sends 100 SYN packets per second, calculate the impact on the server's performance.

Mathematical Formula Bank for Semester Exam

1. **RTT Calculation** (10 Marks)
Formula: $RTT = \frac{1}{f} \times \frac{1}{2}$ where f is the frequency of the signal.
Example: If the frequency is 1000 Hz, calculate the RTT.
Solution: $RTT = \frac{1}{1000} \times \frac{1}{2} = 0.0005$ seconds.

1. Exam Blueprint Matrix

Question Topic	Marks Weightage	Expected Diagram / Formula
3-Way Handshake & SYN Flooding	10 Marks	Timeline diagram + SYN Cookies formula
TCP Header Format	10 Marks	32-bit grid layout + HLEN calculation

TCP Congestion Control (AIMD)	10 Marks	Slow Start & AIMD Sawtooth curve
TCP vs UDP Comparison	10 Marks	10-point architectural table
Token Bucket Numerical	10 Marks	$T = C / (M - R)$ burst calculation